

Projektuppgift

Frontend-baserad webbutveckling

Projektuppgift

Uma Vanmoriel



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Uma Vanmoriel, umva2400@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: VT, 2026

Sammanfattning

Syfte med detta projekt var att skapa en webbapplikation för engelsk språkinlärning som kombinerar innehåll från minst två olika API:er/webbtjänster. Applikationen skulle erbjuda tre funktioner: ordinlärning, textläsning och bokrekommendationer. För att lösa uppgiften användes HTML för strukturen, SASS för styling och JavaScript för funktionalitet. Data hämtades asynkront med Fetch API och async/await. En automatiserad arbetsprocess med Node.js och Parcel användes för att kompilera SCSS till CSS, optimera bilder och möjliggöra live-reloading under utvecklingen. Git användes för versionshantering. Resultatet visar att samtliga krav uppfylldes. För ordinlärningen kombinerades Dictionary API och Unsplash API för att visa ett engelskt ord med definition, uttal och en passande bild. För textläsningen skapades en JSON-fil på GitHub med engelska texter och ordlistor där varje ord hade översättning till svenska och uttal. För bokrekommendationer användes Gutendex API för att visa böcker med titel, författare, omslagsbild och sammanfattning. Webbplatsen är responsiv och fungerar på olika skärmstorlekar. De största utmaningarna under arbetet var att hitta lämpliga API:er som passade uppgiftens syfte, vilket ledde till att en egen JSON-fil fick skapas för textläsningen. Slutsatsen är att trots dessa utmaningar blev slutresultatet en fullt fungerande och användbar webbapplikation för engelsk språkinlärning.

Innehållsförteckning

Sammanfattning

1 Introduktion / Inledning.....	4
2 Teori / Bakgrundsmaterial	5
3 Metod	6
4 Konstruktion	8
5 Resultat	11
6 Slutsatser	13
Källförteckning	14
Bilaga 1: Dokumentation av egenutvecklad programkod	15

1 Introduktion

Projektets syfte är att skapa en webbapplikation för personer som vill lära sig engelska. Applikationen ska hämta och kombinera innehåll från minst två olika webbtjänster/API:er för att skapa ett användbart verktyg för språkinlärning.

Projektet ska ha en startsida som presenterar sidans syfte samt ha länkar till tre alternativ för språkinlärning: 1) ordinlärning, 2) textläsning och 3) bokrekommendationer. Data för dessa sidor ska hämtas från olika API:er. Huvudproblemet blir att kombinera flera API:er så att de tillsammans bygger ett fungerande verktyg för språkinlärning.

Sidan för ordinlärning ska hämta ett nytt ord på engelska, en bild som representerar ordet, en förklaring av ordets betydelse på engelska samt ordets uttal. Utmaningen blir att hitta och kombinera två olika API:er: ett för ord och definitioner (Dictionary API) och ett för bilder (Unsplash API). Användaren ska kunna trycka på en knapp "Lär dig nytt ord" och få ett nytt ord varje gång.

Sidan med textläsning ska hämta en engelsk text med tillhörande ordlista från en JSON-fil. För varje text ska 10 ord från texten presenteras med uttal och översättning till svenska.

För den tredje sidan ska en bokrekommendationsfunktion implementeras med information om engelska böcker från API.

Utöver dessa problem tillkommer utmaningar med att skapa en responsive design med SASS, implementera animeringar, samt att använda en automatiserad arbetsprocess för optimering och publicering.

2 Teori

Webbutveckling börjar oftast med att bestämma målgrupp. Det är viktigt att definiera målgrupp utifrån till exempel ålder, kön eller geografi. Det hjälper till att skapa en relevant webbsida och nå fler användare [1].

Webbsidans struktur ska byggas med HTML (HyperText Mark Language) som är ett standardspråk för att skapa webbsidor [2]. Webbsidan ska designas med SASS (Syntactically Awesome Stylesheets) eller SCSS vilka är preprocessorer som utökar CSS (Cascading Style Sheets). CSS är ett språk som används för att formge sidans utseende exempelvis färger, typsnitt och layout. SCSS använder variabler, nesting och mixins vilket är bra för stora projekt [3].

För att skapa funktionalitet används JavaScript. JavaScript gör webbsidan dynamisk och interaktiv [4].

Node.js är en plattform som gör att JavaScript kan köras på datorn inte bara i webbläsaren [5]. Det gör att man kan använda olika utvecklingsverktyg som till exempel Parcel. Parcel hjälper till att kompilera SCSS till CSS så att webbläsaren förstår den [6]. Det optimerar också bilder så att de blir mindre och laddas snabbare. Parcel skapar även en utvecklingsserver som uppdaterar sidan automatiskt när man sparar ändringar. Genom att använda Node.js och Parcel blir det enklare och snabbare att bygga webbplatsen.

För att göra sidan mer dynamisk används även AJAX (Asynchronous JavaScript and XML) eller Fetch API. AJAX är en teknik för att hämta data från servern asynkront utan att ladda om sidan [7]. Fetch API är en modernare metod för att hämta data och har ersatt XMLHttpRequest [8]. Skillnaden mellan dessa två metoder är att AJAX är den äldre metoden som använder XMLHttpRequest. Den bygger på callback-funktioner och kräver ofta mer kod. Fetch API är den nyare och modernare metoden som använder Promises tillsammans med async/await. Den är enklare, renare och kräver mindre kod [9].

Fetch API hämtar data i formatet JSON (JavaScript Object Notation) från en URL. JSON används för att skicka data från backend till frontend. Den huvudsakliga datastrukturen består av objekt och arrayer [10].

3 Metod

Först ska målgruppen bestämmas. Utifrån den ska webbplatsens design, grafiska profil och innehåll byggas.

För att lösa uppgiften har flera olika metoder och tekniker använts. Eftersom applikationen behöver hämta data från flera olika källor kommer JavaScripts inbyggda Fetch API att användas. Fetch API är en modern metod för att göra asynkrona anrop och kommer att hanteras med `async/await` för att göra koden ren och läsbar [9].

För sidan med ordinläring planeras två API:er att kombineras: Dictionary API för ord, definitioner och uttal samt Unsplash API för bilder. Genom att först hämta ett ord från Dictionary API och sedan använda samma ord för att hämta bilden från Unsplash API planeras att skapa ett kort med engelskt ord, ordets definition och uttal samt bild som representerar ordet.

För att bygga sidan med textläsning planeras en JSON-fil på GitHub att skapas i form av en array som innehåller minst 10 objekt. Varje objekt ska ha id, text på engelska och ordlista med tio ord på engelska med uttal och översättning till svenska. När datan är hämtad ska texten visas tillsammans med en tabell över ordlistan. Användaren ska kunna klicka på en knapp "Nästa text" för att hämta en ny slumpmässig text.

För sidan med bokrekommendationer kommer Gutendex API att användas som innehåller engelska böcker och returnerar data i JSON-format. För att snabba upp laddningen planeras att begränsa antalet böcker till tio per anrop. När sidan laddas ska böcker hämtas och en slumpmässig bok visas med titel, författare, omslagsbild och sammanfattning. En knapp "Nästa bok" ska göra att en ny slumpmässig bok hämtas och visas.

För att lösa problemet med responsiv design och animeringar kommer SCSS med `@media` queries för olika skärmstorlekar att användas [11]. Dessutom planeras en animering med SCSS eller JavaScript för dropdown-menyn för mobilskärmstorlek att skapas, eller någon form av loading spinner att skapas.

En automatiserad arbetsprocess kommer att användas för att optimera och publicera webbplatsen. Detta inkluderar kompilering av SASS till CSS, optimering av bilder och publicering via Git.

Git kommer att användas för versionshantering under hela projektet. Regelbundna commits planeras när nya funktioner läggs till eller buggar

Projektuppgift
Uma Vanmoriel
fixas.

2010-07-05

4 Konstruktion

Målgruppen för denna webbapplikation är nybörjare som vill lära sig engelska. Användarna har ingen specifik ålder eller utan är främst de som vill bygga upp sitt grundläggande ordförråd och förbättra sin läsförståelse. Utifrån denna målgrupp har designen och den grafiska profilen skapats i Figma som visas i Bilaga1 (Bilaga 1).

Webbplatsen byggdes med flera HTML-sidor. Varje sida har en specifik uppgift. För att hålla koden organiserad gjordes separata JavaScript-filer för varje funktion. För "ordinläring"-sida skapades filerna vocabular.js och new-word.js. För "textläsning"-sida skapades reading.js och text.js. För "bokrekommendationer"-sida skapades books.js. Dessutom skapades main.js för gemensamma funktioner som meny och navigering.

För att lösa problemet med att kombinera två API:er för ordinläringssidan skapades en lösning där användaren klickar på knappen "Börja" och navigeras till "new-word.html". När "new-word.html" laddas anropas en funktion som väljer ett slumpmässigt ord från en array i Dictionary API med fetch-anrop. När orddatan är hämtad används ordet för att göra ett nytt fetch-anrop till Unsplash API, där ordet fungerar som sökterm för att hämta en relevant bild. På detta sätt kopplas de två API:erna samman genom att ordnamnet skickas vidare från Dictionary API-svaret till Unsplash API-anropet.

När all data är hämtad skapas HTML-element för att visa ordets namn som en rubrik, uttal som underrubrik, en bild från Unsplash API och en lista med definitioner från Dictionary API. Dessutom skapas en ljudspelare dynamiskt med HTML:s audio-element, där ljudkällan hämtas från Dictionary API:s media-URL för uttal. Knappen "Nytt ord" har en eventlyssnare som gör att samma process körs igen för att hämta ett nytt slumpmässigt ord och uppdatera sidan.

För textsidan skapades först en JSON-fil som laddades upp på GitHub. Filen innehåller en array med minst tio objekt, där varje objekt har id, titel, textinnehåll och en ordlista med tio ord på engelska med översättning till svenska och uttal. När användaren klickar på "Börja läsa" navigeras de till text.html. En funktion anropar sedan en annan funktion som hämtar JSON-filen från GitHub med fetch. När datan är hämtad bearbetas den genom att först välja en slumpmässig text från arrayen. Därefter skapas en rubrik med textens titel och ett stycke med textinnehållet.

En HTML-tabell byggs sedan dynamiskt där funktionen loopar igenom ordlistan och skapar en tabellrad för varje ord med engelska, svenska och uttal. Tabellen skapas med createElement för att ge full kontroll över strukturen, och varje rad får en CSS-klass för styling. Knappen "Nästa text" har en eventlyssnare som gör att samma process körs igen för att hämta och visa en ny slumpmässig text.

För bokrekommendationer användes Gutendex API. En funktion hämtar data från API:et med fetch och begränsar till tio böcker per sida för snabbare laddning. När datan är hämtad anropas en annan funktion som väljer en slumpmässig bok från arrayen med böcker och rensar tidigare innehåll i container-elementet. Därefter skapas en rubrik med bokens titel, följt av en lista där funktionen loopar igenom författarna och skapar listobjekt med författarnamn. Om omslagsbild finns tillgänglig i API-svaret läggs den till, och slutligen skapas ett stycke med bokens sammanfattning.

Under tiden data hämtas visas en laddningsindikator med en CSS-animerad spinner, vilket ger användaren feedback att något händer. Eftersom laddningen av data kan vara långsam och användaren annars kan tro att sidan inte fungerar, är detta viktigt för användarupplevelsen. Knappen "Nästa bok" anropar samma process för att hämta och visa en ny slumpmässig bok.

För att skapa en responsive meny skapades en header med en logo och en navigationsmeny. På större skärmar visas menyn som en horisontell lista, medan en meny ikon visas på mindre skärmar som användaren kan klicka på. I main.js skapades en eventlyssnare som lyssnar efter klick på menyikonen, och när användaren klickar läggs en CSS-klass till på dropdown-menyn som gör att den visas med en CSS-animerad övergång.

För att optimera bilder på "reading"- och "vocabulary"-sidorna användes HTML:s picture-element. Beroende på skärmstorlek och webbläsarstöd laddas olika bildformat som AVIF och WebP samt olika storlekar. Detta minskar laddningstiden och förbättrar webbläsarens prestanda.

SCSS användes för att styla webbplatsen och koden delades upp i fyra filer för att hålla den organiserad. Reset-filen tar bort alla standardstilar från webbläsaren så att sidan ser likadan ut oavsett vilken webbläsare som används. Base-filen innehåller alla färger och typsnitt som används på sidan. Typsnitten Inter och Playfair Display användes som hämtades från Google Fonts. Genom att använda variabler för typsnitt, färger och så vidare blev det lätt att ändra dem senare på en gång utan att ändra en i taget.

Projektoppgift

Uma Vanmoriel

2010-07-05

Component-filen innehåller delar som återanvänds på flera sidor, såsom header, footer, knappar och laddningsspinnern. Layout-filen innehåller stilar för sidornas layout och media queries användes för en responsiv design för olika skärmstorlekar.

Under hela utvecklingen användes Git för versionshantering, och regelbundna commits gjordes när nya funktioner lades till. För den automatiserade arbetsprocessen användes Node.js och Parcel. Parcel kompilerade SCSS till CSS, optimerade bilder och publicerade webbplatsen. Detta gjorde arbetet smidigare eftersom många steg sköttes automatiskt.

5 Resultat

En startsida har skapats med en rubrik som presenterar webbplatsens syfte och en kort beskrivning av vad användaren kan göra. Sidan innehåller tre block med länkar till ordinläring, textläsning och bokrekommendationer. En header med logotyp och navigeringsmeny samt en footer finns på sidan.

Sidan för ordinläring löser problemet med att hämta ett nytt engelskt ord, visa en bild som representerar ordet, visa en förklaring av ordets betydelse på engelska samt visa ordets uttal. Ordet hämtas från Dictionary API via fetch där ett slumpmässigt ord väljs från lista ord. Bilden hämtas från Unsplash API med ordet som sökterm och visas på sidan. Definitioner från Dictionary API visas som en punktlista. Uttal visas både som text och som ljud med audio-spelare där ljudfilen hämtas från Dictionary API. En knapp med texten "Nytt ord" finns på sidan. Vid klick anropas samma funktion som vid sidladdning och ett nytt slumpmässigt ord hämtas och visas. De två API:erna har kombinerats genom att ordet från Dictionary API används som sökterm i Unsplash API vilket innebär att de kommunicerar med varandra.

Sidan för textläsning löser problemet med att hämta en text på engelska och visa tio ord från texten med uttal och översättning till svenska. Texterna hämtas från en JSON-fil på GitHub som innehåller en array med minst tio textobjekt. Textens titel visas som rubrik och textens visas som ett stycke. En tabell skapas dynamiskt med kolumner för engelska, svenska och uttal. En knapp med texten "Nästa text" finns på sidan. Vid klick hämtas en ny slumpmässig text från JSON-filen och sidan uppdateras med nytt innehåll.

Sidan för bokrekommendationer löser problemet med att implementera en bokrekommendationsfunktion med information om engelska böcker. Böcker hämtas från Gutendex API med fetch. Bokens titel visas som rubrik, författarnamn visas som en punktlista, omslagsbild visas i JPEG-format och bokens sammanfattning visas som ett stycke. En knapp med texten "Nästa bok" finns på sidan. Vid klick hämtas en ny slumpmässig bok och en laddningsspinner medan data hämtas från API:et för att ge användaren feedback.

Webbplatsen har en responsive design som anpassar layouten för olika skärmstorlekar. Detta har lösts genom att använda SCSS med @media queries. Flera animeringar har implementerats inklusive en

Projektuppgift

Uma Vanmoriel

2010-07-05

CSS-animerad laddningsspinner, en CSS-animerad dropdown-meny för mobilnavigation samt hover-effekter på knappar och länkar.

En automatiserad arbetsprocess har använts för att kompilera SCSS till CSS, optimera bilder och publicera webbplatsen. Git har använts under hela projektet för att hantera källkod och dokumentation. Regelbundna commits har gjorts och kodhistoriken finns dokumenterad.

Slutligen använder webbplatsen minst två API:er som kommunicerar med varandra som Dictionary API och Unsplash API för "ordinlärning-sida. Dessutom JavaScript-koden är dokumenterad med JSDoc.

6 Slutsatser

Utvecklingen av webbapplikationen gick som planerad i större delen och fungerar som man förväntat sig.

Det som gick bra var att använda Node.js och Parcel för automatiserad arbetsprocess. Det var mycket bekvämt att kunna se ändringar direkt i webbläsaren utan att manuellt behöva kompilera kod eller uppdatera sidan.

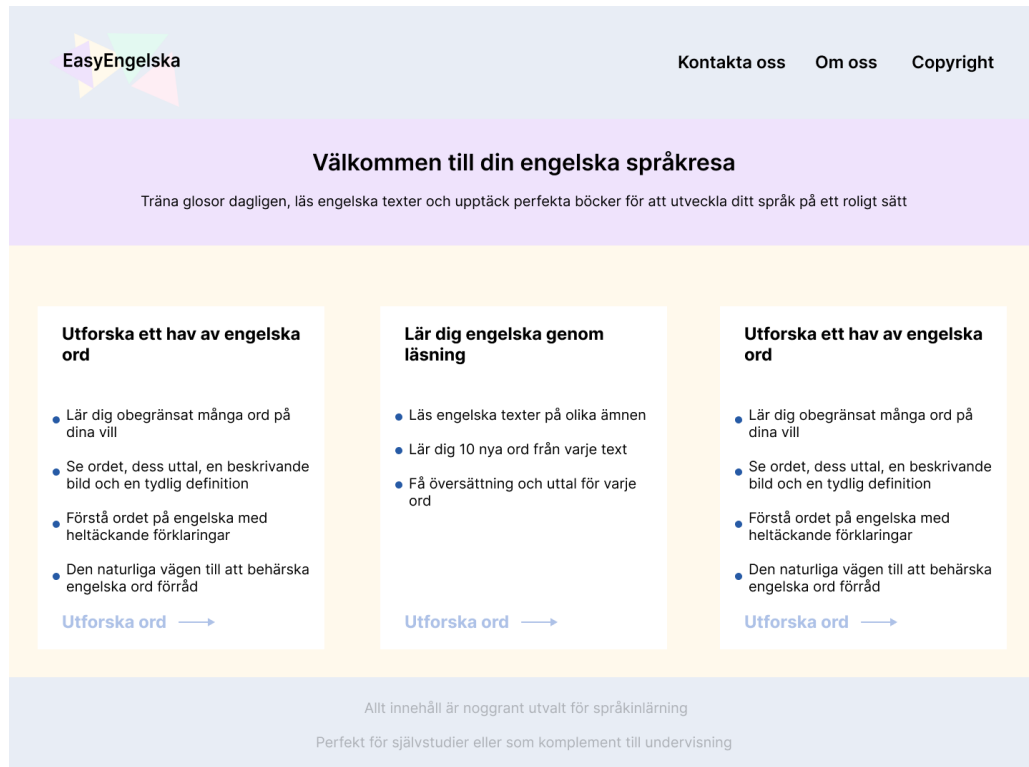
Det som gick dåligt var att hitta lämpliga API:er som passade uppgiftens syfte. Vissa API:er krävde betalning medan andra hade komplicerade implementationer. På grund av detta skapades en egen JSON-fil på GitHub som kunde användas på "textläsning"-sidan. En annan utmaning var att få Unsplash API att hitta relevanta bilder till alla ord så att bilderna stämde med ordets betydelse.

Det som kunde göras annorlunda är att först hitta gratis och fungerande API:er och sedan utifrån dessa planera vilken idé som ska genomföras för att uppfylla uppgiftens krav.

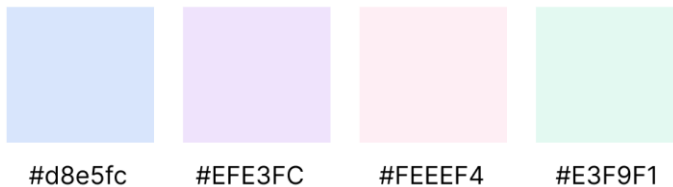
Källförteckning

- [1] Magnus Äng, "Gör Målgruppsanalys & Definiera Målgrupp"
<https://magnusang.se/digital-marknadsforing/malgrupp/> Publicerad 2021-03-24. Hämtad 2026-03-15
- [2] MDN, "Structuring documents" https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Structuring_documents Publicerad 2025-08-25. Hämtad 2026-03-15
- [3] Geekforgeeks, "What is the difference between SCSS and SASS?"
<https://www.geeksforgeeks.org/css/what-is-the-difference-between-scss-and-sass/> Publicerad 2025-03-25. Hämtad 2026-03-15
- [4] Daytona, "Vad är JavaScript och varför är det viktigt för webbutveckling?" <https://daytona.se/vad-ar-javascript-och-varfor-ar-det-viktigt-for-webbutveckling/> Publicerad 2024-05-07. Hämtad 2026-03-15
- [5] Kinsta, "Vad Node.js innebär och varför du ska använda detta"
<https://kinsta.com/se/blog/vad-node-js/> Publicerad 2025-10-01. Hämtad 2026-03-15
- [6] Parcel, "Parcel" <https://parceljs.org/> Hämtad 2026-03-15
- [7] Happiness, "Vad är AJAX? Interaktiva webbsidor"
<https://www.happiness.se/artiklar/vad-ar-ajax> Hämtad 2026-03-15
- [8] MDN, "Fetch API" https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API Publicerad 2026-01-08. Hämtad 2026-03-15
- [9] Geekforgeeks, "Difference Between AJAX And Fetch API"
<https://www.geeksforgeeks.org/javascript/difference-between-ajax-and-fetch-api/> Publicerad 2025-08-05. Hämtad 2026-03-15
- [10] Fullstack23, "Vad är JSON" <https://fullstack23-plus.lms.no-dehill.se/article/vad-ar-json> Hämtad 2026-03-15
- [11] MDN, "Using media queries" https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using Publicerad 2025-12-02. Hämtad 2026-03-15

Bilaga 1: Design och grafisk profil



Grafisk material - Easy Engelska



H1 - Inter , 2.125rem bold

H2 - Inter, 1.75rem bold

H3 - Inter, 1.25rem bold

Brödtext - Playfair Display, 1.125rem

Logotyp

